CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in
Approved by | Affiliated to | UGC Autonomous | 14 Programs Accredited by | Grade A++ in | All India Ranking 151-200 Band | Certifications
COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

SOFT COMPUTING

22CAE01N

CASE STUDY REPORT

An Optimization Similarity Fuzzy Inference Method for Traffic Signal Control at an Isolated Intersection

SUBMITTED BY

Name: Ch Pranav

Roll Number: 1601-24-748-099

B.E. CSM-2

V SEMESTER

III YEAR

ACADEMIC YEAR: 2026-2027

SUBMITTED TO

Dr. Y. C. A. Padmanabha Reddy

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY (A) GANDIPET, HYDERABAD - 500075

Certificate

This is to certify that the case study report entitled **An Optimization Similarity Fuzzy Inference Method for Traffic Signal Control at an Isolated Intersection**, submitted for the subject Soft Computing, has been carried out by the following student during the academic year 2026-27 under my guidance and supervision.

Ch Pranav - 1601-24-748-099

Project Guide:

Dr. Y. C. A. Padmanabha Reddy
Associate Professor
Department of CSE (AI&ML)
CBIT, Hyderabad

Acknowledgement

I express my sincere gratitude to everyone who supported this case study. I thank my project guide, Dr. Y. C. A. Padmanabha Reddy sir, for his guidance and encouragement in the Soft Computing course, and for the opportunity to study fuzzy logic and its application to intelligent traffic signal control through a current, real-world research paper.

I thank the Head of the Department and the management of CBIT for providing the resources and an enabling environment to complete this work.

Finally, I am grateful to my family and friends for their unwavering support and motivation.

Abstract

Signalized intersections are among the largest bottlenecks in urban road networks. Fixed-time and simple actuated controllers cannot react well when traffic fluctuates, and machine-learning controllers are often opaque and computationally heavy. Fuzzy logic offers an interpretable middle path, but the classical Mamdani and Sugeno controllers depend on fixed rules and fixed membership functions, which limits how well they adapt to real-time changes in demand.

This case study examines the 2025 *Multimodal Transportation* paper by Esmaeili, Anjomshoae, Shahsavari-Pour, Srisurin and Banomyong, which introduces the Optimization Similarity Fuzzy Inference (OSFI) method for traffic signal control. OSFI interprets fuzzy reasoning as an optimization problem: the conclusion of a fuzzy rule is chosen so that the similarity between the rule's truth table and the observed fact's truth table is maximized. The controller has three modules — a Next Phase Selector (NPS), a two-stage Green Phase Extender (GPE) with 16 + 13 fuzzy rules, and a Switch Phase Module (SPM) — operating on an isolated four-legged intersection modelled as an M/M/1 queuing system. Under moderate and heavy traffic, the paper reports that OSFI reduces average vehicle delay in every phase, by 1.11–5.73% against Mamdani and 0.69–4.84% against Sugeno controllers. This report also critically reviews the vehicle-exit results, which are lower for OSFI in most phases, and documents a Python simulation prototype built to illustrate the workflow. The prototype confirms that adaptive fuzzy control strongly outperforms fixed-time control under heavy traffic (about 31% lower delay) but, being a simplified approximation, does not reproduce OSFI's advantage over Sugeno. The case study supports UN Sustainable Development Goal 11 (Sustainable Cities and Communities).

Keywords: Fuzzy logic, traffic signal control, optimization similarity fuzzy inference, truth tables, isolated intersection, soft computing.

List of Figures

Figure	Title	Page No.
3.1	Isolated four-legged intersection model with four signal phases	8
4.1	Methodology workflow of the OSFI traffic signal control study	9
4.2	Architecture of the OSFI-based fuzzy traffic signal controller	11
4.3	Two-stage OSFI inference inside the Green Phase Extender	11
4.4	Membership functions of the input “Arrival rate” in FC_1	12
4.5	Membership functions of the input “Queue length” in FC_1	12
4.6	Membership functions of “Extension1” (input to FC_2)	12
6.1	Phase-wise (a) average delay and (b) vehicle exits under moderate traffic	18
6.2	Phase-wise (a) average delay and (b) vehicle exits under heavy traffic	19
6.3	Delay reduction achieved by OSFI per phase and scenario	20
6.4	Prototype average delay by controller and scenario	21
6.5	Queue lengths over time, OSFI-inspired controller, heavy traffic	22

List of Tables

Table	Title	Page No.
2.1	Comparison of traffic signal control approaches	5
3.1	Software environment	7
3.2	Traffic scenarios used in the reference paper	8
3.3	Evaluation metrics	8
4.1	Linguistic variables and supports used in FC_1	13
4.2	Mamdani, Sugeno and OSFI fuzzy controllers compared	14
5.1	Python prototype simulation parameters	15
5.2	Prototype arrival rates (vehicles per second)	16
6.1	Results under moderate traffic (arrival rate Medium, queue Long)	18
6.2	Results under heavy traffic (arrival rate High, queue Very Long)	19
6.3	OSFI delay reduction across scenarios	19
6.4	Prototype results (one run per controller, 3600 s)	21
6.5	OSFI-inspired delay change in the prototype (positive = lower delay)	21

Table of Contents

S.No	Title	Page No.
1.	**1. Introduction**	**1**
	1.1 Background and Significance	1
	1.2 Problem Statement	1
	1.3 Role of Soft Computing	1
	1.4 Key Challenges	2
	1.5 Objectives of This Case Study	2
	1.6 Relevance to Sustainable Development Goal 11	2
	1.7 Report Structure	3
2.	**2. Literature Review and Theoretical Study**	**4**
	2.1 Traditional Signal Control	4
	2.2 Machine Learning and Meta-heuristic Approaches	4
	2.3 Comparative Analysis of Fuzzy Signal Control Models	5
	2.3.1 Classical Mamdani and Sugeno Controllers	5
	2.3.2 Type-2 and Optimized Fuzzy Controllers	5
	2.3.3 Hybrid Fuzzy–Learning Controllers	5
	2.4 Synthesis and Research Gap	6
3.	**3. System Requirements and Specifications**	**7**
	3.1 Hardware and Software Requirements	7
	3.1.1 Hardware Specifications	7
	3.1.2 Software and Libraries	7
	3.2 Intersection Model and Traffic Scenarios	7
	3.3 Evaluation Metrics	8
4.	**4. Methodology and System Design**	**9**
	4.1 Methodology Workflow	9
	4.2 Mathematical Foundation of OSFI	9
	4.2.1 Fuzzy Sets, Relations and Implicators	9
	4.2.2 Similarity Measures	10
	4.2.3 OSFI Reasoning versus the Compositional Rule of Inference	10
	4.3 Proposed System: OSFI Traffic Signal Controller	10
	4.3.1 Next Phase Selector (NPS)	11
	4.3.2 Green Phase Extender (GPE)	11
	4.3.3 Switch Phase Module (SPM)	13
	4.4 Controller Comparison	14
5.	**5. Implementation Details and Experimental Setup**	**15**
	5.1 Implementation Environment	15
	5.2 Experimental Protocol	15
	5.2.1 Setup in the Reference Paper	15

S.No	Title	Page No.
	5.2.2 Python Prototype Parameters	15
	5.2.3 Simulation Loop	16
	5.2.4 Prototype Controller Definitions	16
	5.3 Benchmarking and Comparative Controllers	17
6.	**6. Results and Analysis**	**18**
	6.1 Overview	18
	6.2 Performance under Moderate Traffic	18
	6.3 Performance under Heavy Traffic	19
	6.4 Cross-Scenario Comparison	19
	6.5 Critical Analysis	20
	6.5.1 Why OSFI Reduces Delay	20
	6.5.2 Delay–Throughput Trade-off	20
	6.6 Prototype Simulation Results	21
	6.7 Summary of Findings	23
7.	**7. Conclusion and Future Scope**	**24**
	7.1 Conclusion	24
	7.2 Limitations	24
	7.3 Future Scope	25
8.	**8. Appendix I: Full Python Implementation**	**26**
9.	**9. References**	**31**

1. Introduction

1.1 Background and Significance

More than half of the world's population now lives in cities, and rapid urbanization combined with economic growth has led to a sharp rise in vehicle ownership and use [1]. Urban planners must provide efficient mobility while space for new roads is limited and environmental concerns are growing. Making better use of existing infrastructure, especially at bottlenecks, has therefore become a priority.

Signalized intersections are the main bottlenecks of an urban road network. Unlike interchanges and roundabouts, an intersection interrupts traffic: vehicles must stop so that conflicting movements can proceed, which creates queues and delay. Replacing intersections with grade-separated facilities needs space that dense city centres do not have, so intersections will remain a core part of urban traffic systems. Optimizing signal timing is one of the most cost-effective ways to reduce congestion because it requires no new road space [2].

Signal timing is a delicate balance. A green phase that is too long in one direction produces long red phases, longer queues and extra delay on the other approaches. A green phase that is too short wastes a large part of the cycle on start-up and clearance losses, so fewer vehicles clear the intersection. Choosing good green times in real time is therefore central to the efficiency of an isolated intersection.

1.2 Problem Statement

Existing signal-control approaches either ignore real-time demand (fixed-time), react with rigid logic (actuated), rely on fixed fuzzy rules (Mamdani and Sugeno), or are opaque and computationally heavy (machine learning). A controller is needed that adapts green time to real-time queue lengths and arrival rates at an isolated four-way intersection while remaining interpretable and cheap to compute.

1.3 Role of Soft Computing

Approaches to traffic signal control (TSC) include mathematical optimization, meta-heuristics, machine learning and deep learning, and fuzzy logic. Fuzzy logic is one of the most widely adopted. It was first used for a signal controller at an isolated intersection by Pappis and Mamdani in 1977 [3] and has since become a standard tool for designing and evaluating TSC systems.

Fuzzy control suits problems where precise mathematical modelling is difficult. It encodes expert knowledge as linguistic IF–THEN rules (for example, *IF the queue is long AND arrivals are high THEN extend the green a lot*), mimics human reasoning, and is computationally light. Its rules are transparent, so traffic engineers can read, verify and adjust them — a major advantage over black-box learning models in a safety-critical setting. OSFI extends this soft-computing toolbox by improving the inference step itself.

1.4 Key Challenges

- **Fixed rules and membership functions:** once designed, the rule base and membership functions do not change, which limits adaptation to fluctuating demand, mixed vehicle types and incidents.
- **Scalability:** performance degrades when the number of rules grows or when real-time adjustment of rules is required.
- **Inference quality:** the classical compositional rule of inference (CRI) does not explicitly check how well the inferred conclusion matches the ideal behaviour encoded in the rules.
- **Uncertainty in inputs:** variability in arrival patterns and queue measurements introduces ambiguity at several stages of the control process.

1.5 Objectives of This Case Study

- Study the OSFI method and how it reformulates fuzzy reasoning as similarity optimization over truth tables.
- Analyse how OSFI is applied to signal timing and phase sequencing through the NPS, GPE and SPM modules.
- Compare OSFI with Mamdani and Sugeno controllers on average delay and vehicle exits under moderate and heavy traffic.
- Implement a Python simulation prototype to demonstrate the workflow, and critically assess the results.

1.6 Relevance to Sustainable Development Goal 11

This case study targets UN **SDG 11: Sustainable Cities and Communities**. Smarter signal control reduces congestion and delay at urban intersections without building new roads. Lower

delay and shorter queues mean less idling, lower fuel consumption and emissions, and more liveable cities.

1.7 Report Structure

The report is organised as follows. Chapter 2 reviews the literature on TSC and fuzzy control. Chapter 3 lists the system requirements, the intersection model and the evaluation metrics. Chapter 4 presents the mathematics of OSFI and the controller design. Chapter 5 describes the experimental setup of the paper and of the Python prototype. Chapter 6 presents and interprets the results. Chapter 7 gives the conclusion, limitations and future scope. Chapter 8 (Appendix I) lists the full Python implementation, and Chapter 9 lists the references.

2. Literature Review and Theoretical Study

2.1 Traditional Signal Control

Traffic signal control was first deployed in Cleveland, Ohio, in 1913. Webster's 1958 work [4] gave an equation for the optimal cycle length that is still widely used, although later studies found that it produces excessively long cycles under oversaturated conditions. Gazis applied Pontryagin's control theory to oversaturated intersections, Babicheva used queuing theory, and Srisurin and Singh proposed a linear programming model to minimize queues during peak hours [1].

Real-world signal operations fall into three categories [5]:

- **Fixed-time / pre-timed:** cycle length, interval timings and phase sequence are constant, or vary only by time of day. Simple, but delays grow when demand fluctuates.
- **Actuated:** detectors (loops, cameras, sensors) measure flow or presence, and timings change in response.
- **Semi-actuated:** the main street runs on a fixed cycle while minor-street approaches have detectors.

2.2 Machine Learning and Meta-heuristic Approaches

Machine learning. Deep reinforcement learning (DRL) lets controllers learn from data and adapt to changing patterns [6]. Recent work includes graph-based Q-learning for flexible phase actions, co-evolutionary reinforcement learning for networks with stochastic capacity, emission-aware reward functions, heterogeneous graph neural networks combined with DRL, and multi-agent frameworks that coordinate many intersections [1]. These methods are adaptive but act as black boxes and need heavy computation and training data.

Meta-heuristics. Nature-inspired search algorithms explore large solution spaces where exact methods fail. Examples include a differential evolution model for three-leg intersections with up to 42% delay reduction in VISSIM [7], genetic programming that produces interpretable phase-switching rules, and an adaptive artificial fish swarm algorithm for smart-city intersections.

2.3 Comparative Analysis of Fuzzy Signal Control Models

2.3.1 Classical Mamdani and Sugeno Controllers

Pappis and Mamdani [3] introduced the first fuzzy signal controller. Niittymäki and Pursula [8] built an early real-time adaptive fuzzy controller, Murat and Gedizlioglu [9] proposed a multi-phase fuzzy model for isolated junctions, and Collotta et al. [10] combined wireless sensor networks with multiple parallel fuzzy controllers. Fuzzy controllers are transparent, need no precise mathematical model and run cheaply, but they rely on fixed rule bases.

2.3.2 Type-2 and Optimized Fuzzy Controllers

Type-2 fuzzy logic models uncertainty in the membership functions themselves. Bi et al. [11] proposed a multi-agent type-2 fuzzy controller optimized by differential evolution, and Jovanović et al. [12] applied interval type-2 fuzzy logic to restricted crossing U-turn control. Ali et al. [13] combined fuzzy logic with Webster formulas in SUMO, and Lin et al. [14] tuned fuzzy control with a multi-objective differential evolution algorithm to cut delay, fuel use and emissions.

2.3.3 Hybrid Fuzzy–Learning Controllers

Hybrid approaches combine fuzzy logic's handling of imprecision with the optimization or learning power of other methods. Chiou and Huang [15] built a stepwise genetic fuzzy controller for mixed traffic, and Kumar et al. [16] combined fuzzy inference with DRL to reduce waiting time, fuel consumption and emissions. Table 2.1 summarises the approaches.

Table 2.1: Comparison of traffic signal control approaches

Approach	Strength	Limitation
Fixed-time / pre-timed	Simple, cheap, predictable	Preset cycles ignore real demand, so delays grow when traffic fluctuates
Actuated / semi-actuated	Responds to detector input	Logic remains relatively rigid
Mamdani and Sugeno fuzzy	Interpretable, model-free, low compute	Fixed rules and membership functions limit real-time adaptability
Type-2 / optimized fuzzy	Better handling of uncertainty	More parameters; tuning is offline and costly
Meta-heuristics	Search large timing spaces well	Iterative, often offline; limited interpretability
ML / deep RL	Highly adaptive, learns from data	Black box, heavy compute, needs training data, harder to verify
OSFI (this study)	Rule-based and interpretable; inference optimized against truth tables	Evaluated only on an isolated intersection in simulation

2.4 Synthesis and Research Gap

The literature shows a clear trade-off. Learning-based controllers are adaptive but opaque and expensive, while fuzzy controllers are interpretable and light but rigid. Fuzzy logic improves through gradual refinement of rules and membership functions rather than large-scale retraining [17], which keeps it stable and explainable.

Although fuzzy logic has been applied extensively to isolated intersections, there is little research on how advanced inference techniques can improve the adaptability of signal systems. The OSFI method, which optimizes the similarity between predicted and desired outcomes using truth tables as benchmarks [18], [19], had not been rigorously applied to traffic signal control. The reference paper addresses this gap.

3. System Requirements and Specifications

3.1 Hardware and Software Requirements

3.1.1 Hardware Specifications

Simulation (this study). Fuzzy inference is lightweight, so a standard computer is sufficient: any modern dual-core processor, 4–8 GB RAM and about 100 MB of free disk space for the code, results and graphs. No GPU is required.

Field deployment. A real OSFI controller would need sensors to measure queue length and arrival rate on each approach (inductive loop detectors, video cameras or radar), a roadside signal controller or embedded processor to run the inference every cycle, and a communication link to the signal heads.

3.1.2 Software and Libraries

Table 3.1: Software environment

Component	Role
MATLAB	Used in the reference paper for the Next Phase Selector logic and the fuzzy simulation
Python 3	Language of the simulation prototype (traffic_simulation.py, Appendix I)
NumPy	Poisson arrivals, random discharge and array maths
Matplotlib	Graphs of delay, throughput and queue length
csv, pathlib	Writing results.csv and the graphs folder

3.2 Intersection Model and Traffic Scenarios

The study uses an isolated four-legged intersection with left-hand traffic (Figure 3.1). There are eight possible movements, but the controller runs four phases; each phase gives green to the through and right-turn movements of one approach, and only one phase is green at a time. The phases are East (E), South (S), West (W) and North (N).

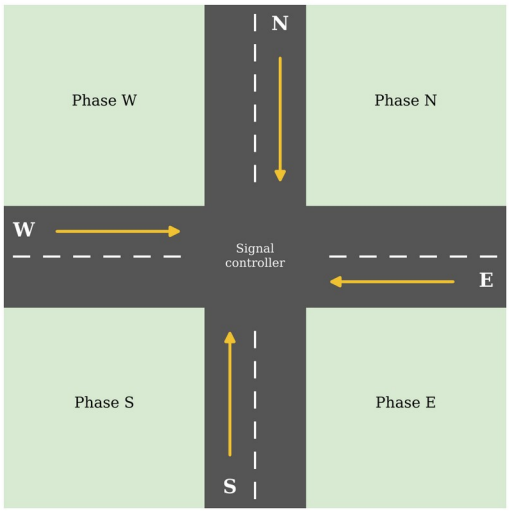


Figure 3.1: Isolated four-legged intersection model with four signal phases

Traffic is modelled as an **M/M/1 queue** [20]. The first M means memoryless (exponential) inter-arrival times, so arrivals follow a Poisson process with mean rate λ vehicles per 10 seconds. The second M means i.i.d. memoryless service (discharge) times, and the 1 means a single server, because only one phase is green at any moment. The intersection is therefore four queues served in turn by one server, with first-in, first-out discipline. Table 3.2 lists the two traffic scenarios.

Table 3.2: Traffic scenarios used in the reference paper

Criterion	Scenario 1: Moderate	Scenario 2: Heavy
Arrival rate (λ)	Medium (M)	High (H)
Queue length	Long (L)	Very Long (VL)
Represents	Regular-hour flow	Peak-hour congestion

3.3 Evaluation Metrics

Table 3.3: Evaluation metrics

Measure	Definition and use
Average delay (s)	Mean time each vehicle waits at the intersection before passing through; lower is better
Average exit (veh)	Number of vehicles discharged per phase during the simulation period (throughput); higher is better
Delay reduction (%)	(B − O) / B × 100, where B is the baseline delay and O is the OSFI delay
Prototype metrics	Average delay, throughput after warm-up, average queue per approach, vehicles left in queue

4. Methodology and System Design

4.1 Methodology Workflow

Figure 4.1 summarises the workflow followed in the reference paper and in this case study, from modelling the intersection to comparing the controllers.

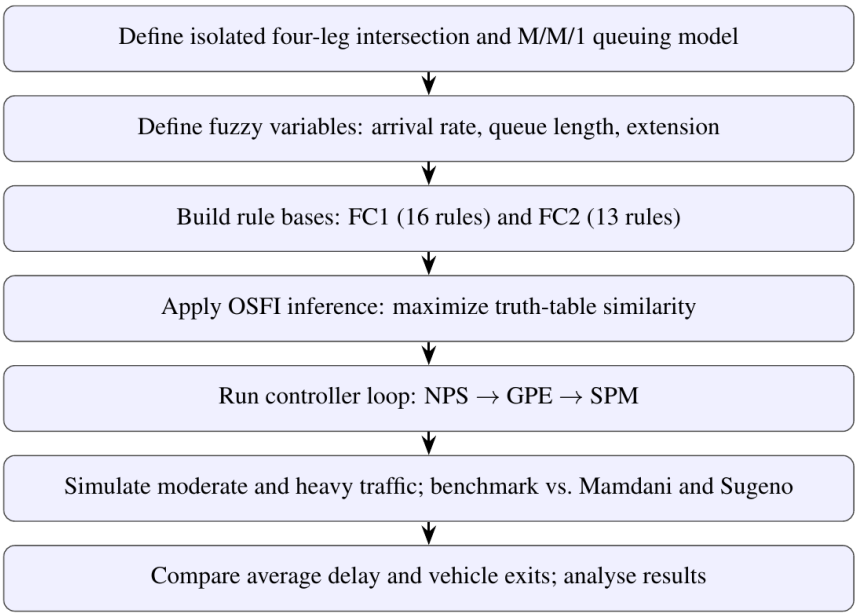


Figure 4.1: Methodology workflow of the OSFI traffic signal control study

4.2 Mathematical Foundation of OSFI

OSFI builds on the method of optimizing the similarity of truth tables introduced by Deng and Jiang [18] and extended by Esmaeili and Eslami [19]. The definitions below follow the reference paper [1].

4.2.1 Fuzzy Sets, Relations and Implicators

Definition 1 (Fuzzy set). For a fixed crisp set E, a fuzzy set M in E is defined as follows, where M(e) is the degree of membership of e in M [21]; FS(E) denotes the set of all fuzzy sets on E:

$$M = \{ (e, M(e)) \mid e \in E \}, \quad M(e) : E \rightarrow [0, 1]$$

Definition 2 (Fuzzy relation). For finite non-empty sets E_1 and E_2, a fuzzy relation R_e from E_1 to E_2 is:

$$R_e = \{ ((e_1, e_2), R_e(e_1, e_2)) \mid e_1 \in E_1, e_2 \in E_2 \}, \quad R_e : E_1 \times E_2 \rightarrow [0, 1]$$

Definition 3 (Fuzzy implicator). A fuzzy implicator on [0, 1] is a mapping $I : [0, 1] \times [0, 1] \rightarrow [0, 1]$ with I(0, 0) = 1, I(1, 0) = 0, I(0, 1) = 1 and I(1, 1) = 1 [22]. For all a, b ∈ [0, 1], common implicators are:

- **Mamdani:** $I_{min}(a, b) = \min(a, b)$
- **Łukasiewicz:** $I_{Lu}(a, b) = \min(1, 1 - a + b)$
- **Goguen:** $I_G(a, b) = 1$ if $a \le b$, and b / a if $a > b$
- **Zadeh:** $I_Z(a, b) = \max(1 - a, \min(a, b))$

4.2.2 Similarity Measures

Definition 4 (Similarity measure). A similarity measure on FS(E) is a function $S_e : FS(E) \times FS(E) \to [0, 1]$ such that, for all $E_1, E_2, E_3 \in FS(E)$: (1) $0 \le S_e(E_1, E_2) \le 1$; (2) $S_e(E_1, E_2) = S_e(E_2, E_1)$; (3) $S_e(E_1, E_2) = 1$ if and only if $E_1 = E_2$; and (4) if $E_1 \subseteq E_2 \subseteq E_3$, then $S_e(E_1, E_3) \le S_e(E_1, E_2)$ and $S_e(E_1, E_3) \le S_e(E_2, E_3)$. Similarity relations of this kind were formalized by Zadeh [23].

4.2.3 OSFI Reasoning versus the Compositional Rule of Inference

Consider the fuzzy modus ponens problem — Rule: $E_A(x) \to E_B(y)$; Fact: $E_A^*(x)$; Conclusion: $E_B^*(y)$ — where E_A, E_A^* are fuzzy sets on $X = \{x_1, \ldots, x_n\}$, E_B, E_B^* are fuzzy sets on $Y = \{y_1, \ldots, y_m\}$, and $\to$ is a fuzzy implicator.

(1) Compositional rule of inference (CRI). Zadeh's CRI [24] first converts the rule into a fuzzy relation, the *truth table*, and then composes it with the fact:

$$R_e(x, y) = I(\, E_A(x), E_B(y)\,), \qquad E_B^*(y) = \bigvee_{x \in X} [\, E_A^*(x) \wedge R_e(x, y)\,]$$

(2) OSFI. Instead of composing, OSFI treats inference as an optimization problem. The conclusion E_B^* is the fuzzy set whose truth table is most similar to the truth table of the original rule:

$$\text{maximize } \Phi(E_B^*) = \mathrm{Sim}(\, E_A \to E_B\,,\; E_A^* \to E_B^*\,), \quad E_B^* \in FS(Y) \qquad (4.1)$$

Here Sim is a fuzzy similarity measure, and the optimal E_B^* of Problem (4.1) is the inferred conclusion [18]. Intuitively, if the observed fact is close to the rule's antecedent, the conclusion is pushed to be close to the rule's consequent, so the controller stays aligned with the ideal input–output behaviour defined by the rule base. Because it remains a rule-based refinement rather than a learned model, OSFI keeps the interpretability and stability of fuzzy logic [17].

4.3 Proposed System: OSFI Traffic Signal Controller

The controller consists of three modules, shown in Figure 4.2. **Step A – NPS:** input the queue lengths of the four phases and output the next phase. **Step B – GPE:** input the arrival rate and queue length of the selected phase, apply the fuzzy rule base with OSFI, and output the green-time extension. **Step C – SPM:** combine the outputs of Steps A and B and switch the phase.

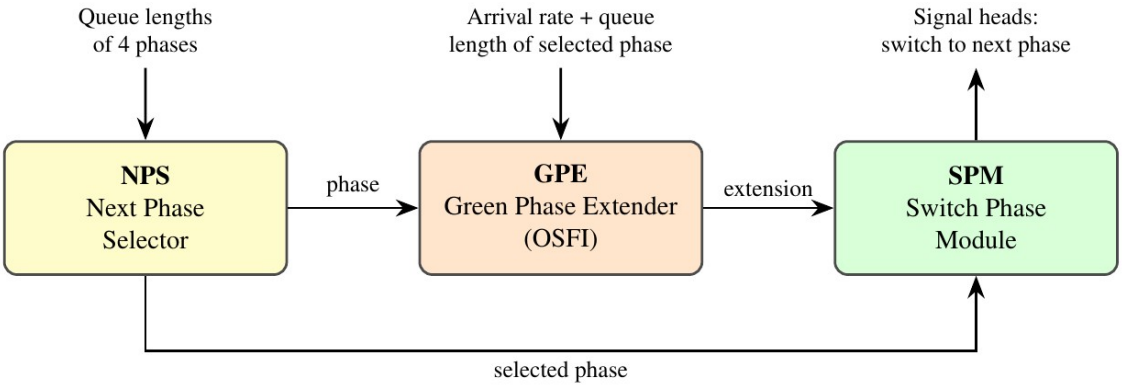


Figure 4.2: Architecture of the OSFI-based fuzzy traffic signal controller

4.3.1 Next Phase Selector (NPS)

The NPS decides which phase should receive green next. It reads the queue length from the sensor on each of the four approaches and selects the phase with the longest queue. In the paper this is implemented in MATLAB: the East queue is compared with the other three; if it is the longest, phase E is selected, otherwise the comparison continues among S, W and N until the longest queue is found.

4.3.2 Green Phase Extender (GPE)

The GPE decides how long to extend the green of the selected phase using a two-stage fuzzy inference system (Figure 4.3). FC_1 computes an initial extension from the arrival rate and queue length; FC_2 refines it using the queue lengths of the other three phases.

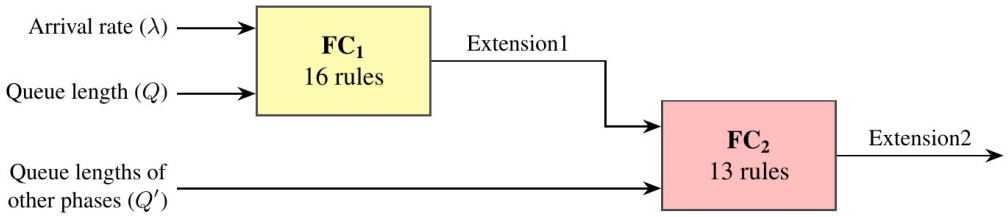


Figure 4.3: Two-stage OSFI inference inside the Green Phase Extender

FC_1 (16 rules). Inputs are *Arrival rate* with four fuzzy sets — Zero (Z), Low (Lo), Medium (M) and High (H) — over 0–12 vehicles per 10 s (Figure 4.4), and *Queue length* with four fuzzy sets — Short (Sh), Medium (M), Long (L) and Very Long (VL) — over 0–60 m (Figure 4.5). With 4 × 4 = 16 combinations the rule base is complete. The output *Extension1* has the fuzzy sets Zero, Short, Medium and Long over 0–25 s (Figure 4.6). Table 4.1 lists the supports of all FC_1 variables. Example rules:

- Rule 1: IF Arrival rate is Zero AND Queue length is Short THEN Extension1 is Zero.
- Rule 2: IF Arrival rate is Low AND Queue length is Long THEN Extension1 is Short.

- Rule 3: IF Arrival rate is Medium AND Queue length is Short THEN Extension1 is Short.

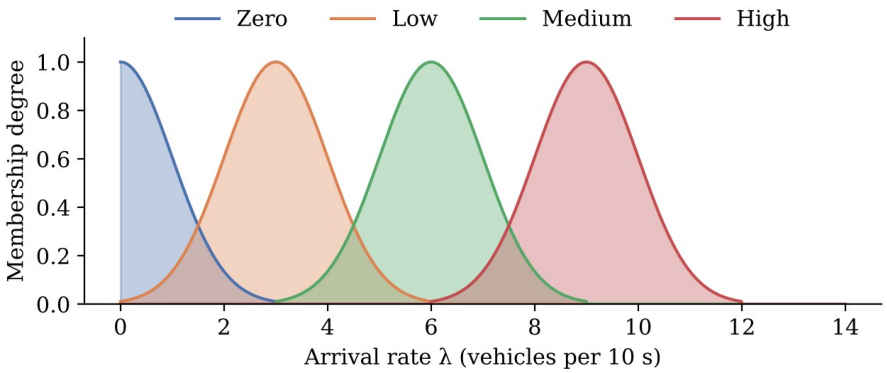


Figure 4.4: Membership functions of the input "Arrival rate" in FC_1

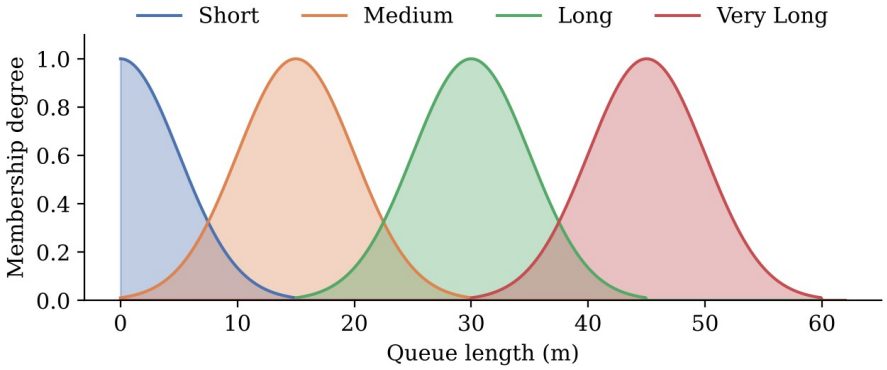


Figure 4.5: Membership functions of the input "Queue length" in FC_1

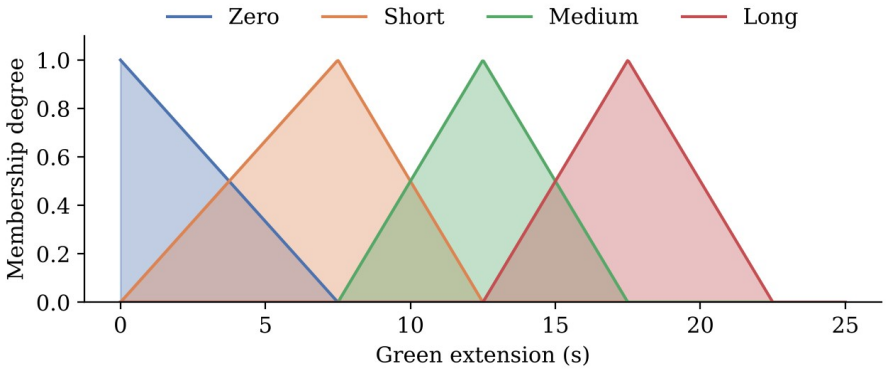


Figure 4.6: Membership functions of "Extension1" (input to FC_2)

Table 4.1: Linguistic variables and supports used in FC_1

Variable	Zero / Short	Low / Medium	Medium / Long	High / Very Long
Arrival rate λ (veh / 10 s)	Z: [0, 3]	Lo: [0, 6]	M: [3, 9]	H: [6, 12]
Queue length (m)	Sh: [0, 15]	M: [0, 30]	L: [15, 45]	VL: [30, 60]
Extension1 (s)	Z: [0, 7]	Sh: [0, 13]	M: [7, 17]	L: [13, 25]

FC_2 (13 rules). Inputs are *Extension1* and the queue lengths of the other three phases. The output *Extension2* (Zero, Short, Medium, Long) is the final green extension; this stage reduces the extension when other approaches are building long queues. Example rules:

- IF Extension1 is Zero AND Queue length is Short THEN Extension2 is Zero.
- IF Extension1 is Short AND Queue length is Medium THEN Extension2 is Short.
- IF Extension1 is Medium AND Queue length is Short THEN Extension2 is Medium.

Rule firing and output. Both fuzzy systems use fixed, crisp output levels $z_i = c_i$. The firing strength of rule i is $\alpha_i = \text{AND}(f_1, f_2)$, where the AND operation is implemented with the Łukasiewicz implicator; f_1 and f_2 are the memberships of the two inputs. The system output is the weighted average of the N rule outputs, and centroid defuzzification gives a crisp extension that can be applied directly to the signal:

$$z^* = (\Sigma_{i=1..N} \alpha_i z_i) / (\Sigma_{i=1..N} \alpha_i) \qquad (4.2)$$

4.3.3 Switch Phase Module (SPM)

The SPM combines the NPS and GPE outputs. If another phase has a longer queue than the present phase, the SPM switches green to that phase, and the GPE determines how long the next green will be by considering the conditions of the other phases. Because the green time of each phase adapts to demand, the cycle length is flexible rather than fixed.

4.4 Controller Comparison

Table 4.2: Mamdani, Sugeno and OSFI fuzzy controllers compared

Feature	Mamdani (CRI)	Sugeno	OSFI
Inference	Compositional rule of inference (min/max)	Weighted average of crisp rule outputs	Maximize similarity of truth tables
Rule output	Fuzzy set	Crisp constant / function	Fuzzy set chosen by optimization
Adaptation	Fixed rules, manual tuning	Fixed rules, manual tuning	Rule consistency optimized against truth-table benchmarks
Interpretability	High	High	High
Computation	Low	Lowest	Moderate (optimization step)
Defuzzification	Centroid	Centroid	Centroid

5. Implementation Details and Experimental Setup

5.1 Implementation Environment

The reference paper implements the controller in MATLAB and evaluates it by traffic-flow simulation on the isolated intersection, using a mix of synthetic and real-world traffic data. For this case study, a separate Python prototype, traffic_simulation.py (listed in full in Appendix I), was written to illustrate the workflow. It is a time-stepped simulation of a four-way intersection that compares four controllers: fixed-time, Mamdani-style fuzzy, Sugeno-style fuzzy and an OSFI-inspired similarity-tuned fuzzy controller.

The prototype is an **educational approximation, not an exact reproduction** of the paper. The paper does not publish all membership-function parameters, the full rule bases or the original truth table, so the prototype uses transparent, illustrative definitions.

5.2 Experimental Protocol

5.2.1 Setup in the Reference Paper

- Isolated four-way intersection with phases E, S, W and N, one green at a time.
- M/M/1 queuing model with Poisson arrivals (rate λ per 10 s) and FIFO discipline.
- Two scenarios: moderate (arrival Medium, queue Long) and heavy (arrival High, queue Very Long).
- The same two-stage inference structure (Figure 4.3) and centroid defuzzification for all three controllers, so the comparison is consistent.
- Metrics: average delay (s) and average exit (vehicles) per phase.

5.2.2 Python Prototype Parameters

Table 5.1: Python prototype simulation parameters

Parameter	Value	Meaning
SIM_TIME	3600 s	Total simulated time (one hour)
WARMUP	300 s	Initial period excluded from metrics
MIN_GREEN / MAX_GREEN	8 s / 35 s	Bounds on green duration
BASE_GREEN	10 s	Base green before the extension is added
CLEARANCE	2 s	Simplified all-red time between phases
VEHICLES_PER_SECOND	0.90	Mean discharge capacity during green
SEED	42	Base random seed

Table 5.2: Prototype arrival rates (vehicles per second)

Scenario	North	East	South	West
Moderate	0.10	0.12	0.09	0.11
Heavy	0.18	0.20	0.16	0.19

5.2.3 Simulation Loop

Each simulated second, the prototype performs the following steps:

1. Adds a Poisson-distributed number of vehicles to each approach, storing their arrival times.
2. When the current green and clearance have ended, selects the next phase: fixed-time rotates, while the adaptive controllers pick the longest queue (playing the role of the NPS).
3. Computes green time as BASE_GREEN + extension, clipped to 8–35 s (fixed-time always uses 15 s).
4. Discharges vehicles from the active queue at a mean of 0.90 vehicles/s and records waiting time after warm-up.
5. Inserts a 2 s clearance before the next phase.
6. Reports average delay, throughput, average queue, total arrivals and vehicles still queued.

5.2.4 Prototype Controller Definitions

- **Fixed-time:** rotates through the phases with a constant 15 s green.
- **Mamdani-style fuzzy:** queue length and arrival rate are mapped to Low/Medium/High by linear and triangular functions. Six rules combine them with the minimum (AND) operator and point to output extensions of 0, 3, 3, 7, 15 or 11 s. Clipped triangular output sets are aggregated and the extension is the centroid.
- **Sugeno-style fuzzy:** same membership functions and rules, but the output is the weighted average of the crisp rule outputs.
- **OSFI-inspired:** a target table defines the desired extension from a demand score 0.75 × queue + 24 × arrival rate (0 s below 6, 4 s below 14, 9 s below 24, 15 s otherwise). A lookup table over 6 queue bins and 5 arrival bins stores the target output, and at run time the nearest cell is used.

5.3 Benchmarking and Comparative Controllers

In the reference paper, OSFI is benchmarked against **Mamdani** (CRI-based) and **Sugeno** fuzzy inference controllers, the two most widely used fuzzy TSC methods, which are known to struggle in highly dynamic conditions. The prototype adds a **fixed-time** controller as a non-adaptive baseline.

6. Results and Analysis

6.1 Overview

This chapter presents the results reported in the reference paper for moderate and heavy traffic, interprets them, and then presents the results of the Python prototype. Delay is in seconds (lower is better); exits are numbers of vehicles discharged per phase.

6.2 Performance under Moderate Traffic

Table 6.1: Results under moderate traffic (arrival rate Medium, queue Long)

Indicator	Phase	Mamdani	Sugeno	OSFI	vs Mamdani	vs Sugeno
Average delay (s)	E	1.048	1.018	**1.011**	3.55%	0.69%
	S	1.210	1.199	**1.182**	2.31%	1.42%
	W	1.172	1.183	**1.153**	1.62%	2.54%
	N	1.062	1.074	**1.022**	3.78%	4.84%
Average exit (veh)	E	328	320	310	5.49%	3.13%
	S	398	397	387	2.76%	2.52%
	W	165	145	138	16.36%	4.83%
	N	173	175	173	0.58%	1.14%

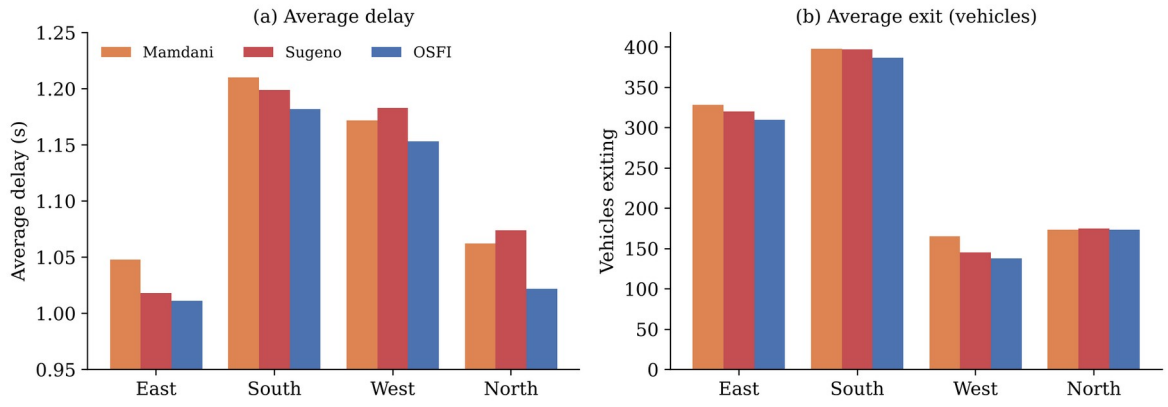


Figure 6.1: Phase-wise (a) average delay and (b) vehicle exits under moderate traffic

Under moderate traffic, OSFI has the lowest delay in all four phases: 1.62–3.78% lower than Mamdani and 0.69–4.84% lower than Sugeno. The largest gain is in the North phase (4.84% against Sugeno).

6.3 Performance under Heavy Traffic

Table 6.2: Results under heavy traffic (arrival rate High, queue Very Long)

Indicator	Phase	Mamdani	Sugeno	OSFI	vs Mamdani	vs Sugeno
Average delay (s)	E	1.137	1.126	**1.098**	3.64%	2.55%
	S	1.178	1.201	**1.165**	1.11%	3.09%
	W	1.172	1.154	**1.132**	3.53%	1.94%
	N	1.144	1.131	**1.082**	5.73%	4.52%
Average exit (veh)	E	347	321	298	16.44%	7.71%
	S	381	385	362	5.20%	6.35%
	W	152	137	128	18.75%	2.34%
	N	186	185	178	4.49%	3.93%

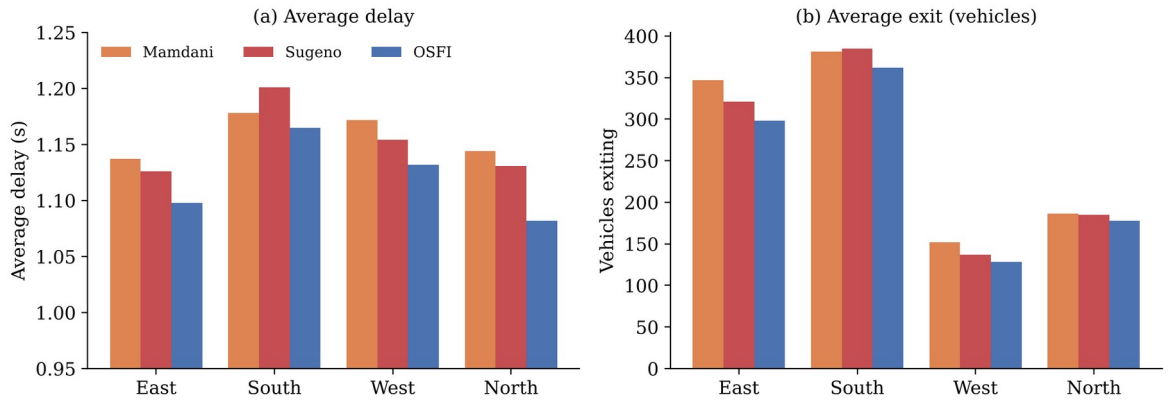


Figure 6.2: Phase-wise (a) average delay and (b) vehicle exits under heavy traffic

Under heavy traffic, OSFI again has the lowest delay in every phase: 1.11–5.73% lower than Mamdani and 1.94–4.52% lower than Sugeno. The largest reduction overall, 5.73%, is in the North phase against Mamdani.

6.4 Cross-Scenario Comparison

Table 6.3: OSFI delay reduction across scenarios

Comparison	Moderate traffic	Heavy traffic
OSFI vs Mamdani	1.62% to 3.78%	1.11% to 5.73%
OSFI vs Sugeno	0.69% to 4.84%	1.94% to 4.52%
Phases where OSFI has the lowest delay	4 of 4	4 of 4

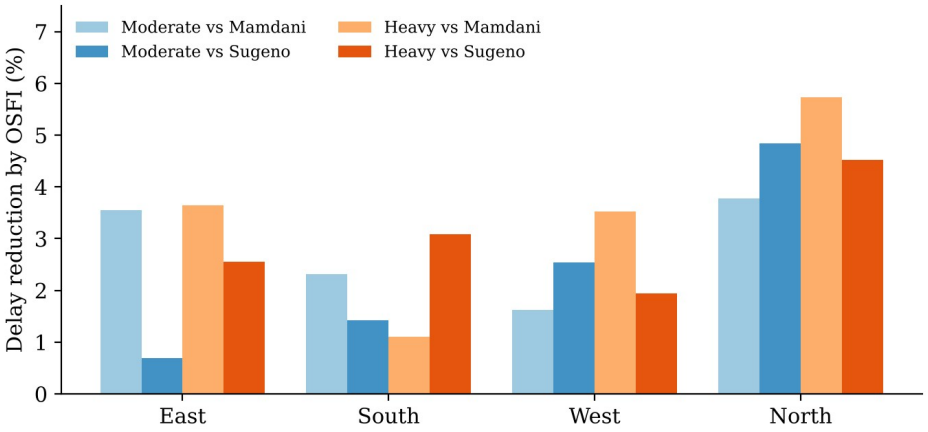


Figure 6.3: Delay reduction achieved by OSFI per phase and scenario

All 16 delay comparisons (4 phases × 2 scenarios × 2 baselines) favour OSFI.

6.5 Critical Analysis

6.5.1 Why OSFI Reduces Delay

- **Inference aligned with ideal behaviour:** CRI composes the fact with the truth table but does not check how well the result matches the rule's intended behaviour. OSFI explicitly chooses the conclusion that maximizes truth-table similarity, so the green extension tracks the ideal response more closely.
- **Two-stage, context-aware extension:** FC_2 limits the extension when other phases have long queues, preventing one direction from starving the others.
- **Flexible cycle length:** green times adapt phase by phase, so the cycle stretches or shrinks with demand instead of following a preset plan.
- **Incremental, stable improvement:** the gains are small (about 1–6%), consistent with OSFI being a refinement of fuzzy reasoning rather than a new control paradigm.

6.5.2 Delay–Throughput Trade-off

The exit results need careful reading. In Tables 6.1 and 6.2, OSFI's exit counts are *lower* than Mamdani's and Sugeno's in most phases (for example, heavy-traffic East: 298 vs 347 and 321). The paper's comparison column reports these differences as improvements of up to 18.75%, but its own phase-wise analysis acknowledges that OSFI shows lower throughput in the East and West phases under heavy traffic. Since higher exits indicate better throughput, the percentages in the exit rows are better read as *differences*, not gains.

A plausible explanation is that OSFI serves shorter, better-matched green extensions: each vehicle waits less, but fewer vehicles are discharged per phase within the simulated period. The

delay gains are real but should not be read as an unqualified improvement; delay and throughput should always be reported together.

6.6 Prototype Simulation Results

The prototype was run once per controller with the settings in Chapter 5. Table 6.4 and Figure 6.4 show the results.

Table 6.4: Prototype results (one run per controller, 3600 s)

Scenario	Controller	Delay (s)	Throughput	Avg. queue	Arrivals	Left in queue
Moderate	Fixed-time	25.89	1367	2.69	1497	8
	Mamdani	22.98	1384	2.41	1519	8
	Sugeno	**20.23**	1428	2.19	1548	8
	OSFI-inspired	24.03	1458	2.65	1564	4
Heavy	Fixed-time	56.23	2415	10.34	2649	38
	Mamdani	43.22	2429	7.95	2640	18
	Sugeno	**36.60**	2402	6.66	2626	21
	OSFI-inspired	38.58	2382	6.95	2613	27

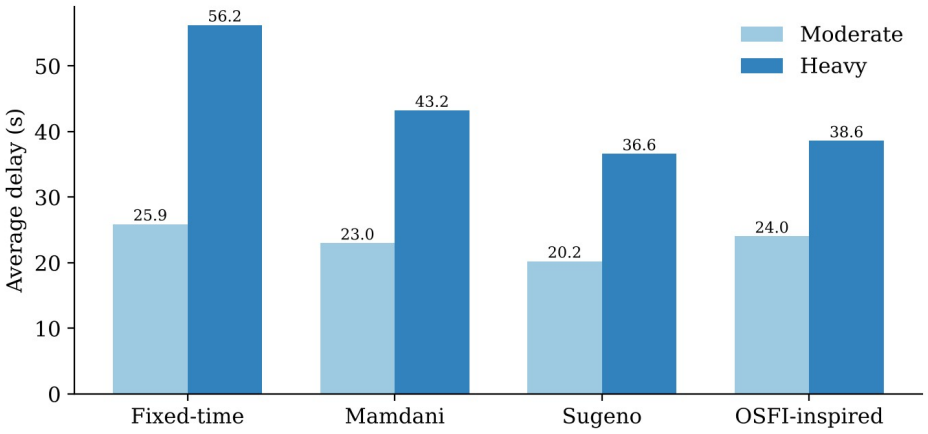


Figure 6.4: Prototype average delay by controller and scenario

Table 6.5: OSFI-inspired delay change in the prototype (positive = lower delay)

Comparison	Moderate	Heavy
vs Fixed-time	+7.2%	+31.4%
vs Mamdani	−4.6%	+10.7%
vs Sugeno	−18.8%	−5.4%

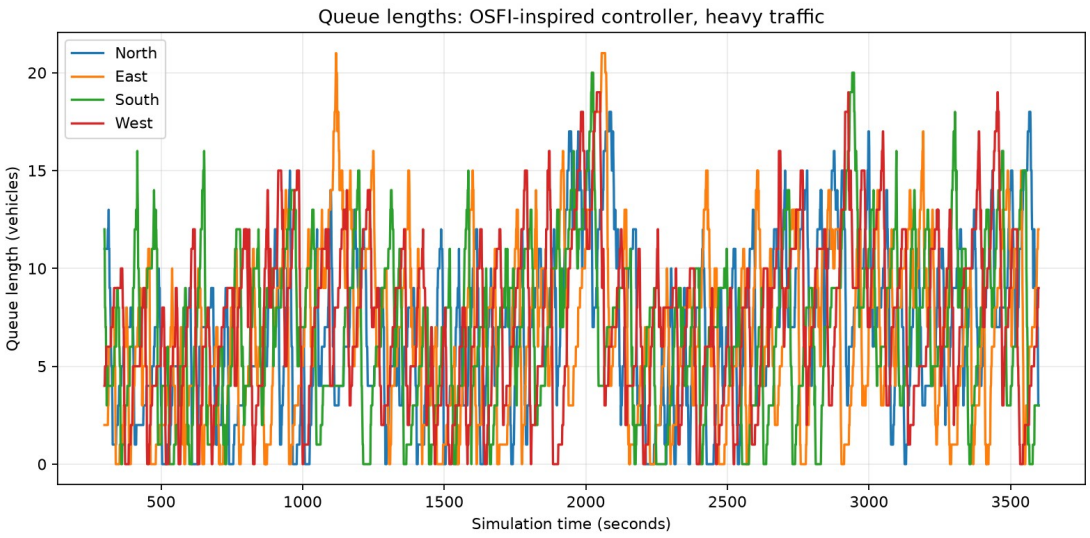


Figure 6.5: Queue lengths over time, OSFI-inspired controller, heavy traffic

All fuzzy controllers beat fixed-time control, slightly under moderate traffic and strongly under heavy traffic, where the OSFI-inspired controller cuts delay by about 31%. It also beats Mamdani by about 11% under heavy traffic. However, the Sugeno-style controller has the lowest delay in both scenarios, so the prototype **does not reproduce** the paper's headline result. The reasons are known simplifications:

1. **No real similarity optimization:** the OSFI-inspired controller copies a heuristic target table instead of maximizing Φ against a truth table.
2. **Unused table range:** arrival bins span 0.10–0.50 veh/s, but the simulated rates are only 0.09–0.20, so the lookup is coarse.
3. **Nearest-cell lookup:** the extension jumps between a few fixed values, while Mamdani and Sugeno vary smoothly.
4. **Constant arrival input:** controllers receive the true mean arrival rate, not a measured one.
5. **Single-stage inference:** one stage with six rules instead of the paper's 16 + 13 rules.
6. **Different scales:** one-second steps with explicit clearance give delays of tens of seconds, not comparable with the paper's values near 1 s.

In addition, the random seed changes with the controller index, so arrival totals differ slightly between controllers (1497–1564 in the moderate scenario). Throughput is therefore not a like-for-like comparison, and a single run is not enough to declare a winner.

6.7 Summary of Findings

- In the reference paper, OSFI gives the lowest average delay in all 16 comparisons, with reductions of up to 5.73%.
- The gains are incremental (about 1–6%) and come with lower exit counts in most phases.
- The prototype confirms that adaptive fuzzy control strongly outperforms fixed-time control, but its simplified OSFI-inspired controller does not beat Sugeno.

7. Conclusion and Future Scope

7.1 Conclusion

This case study examined the optimization similarity fuzzy inference method for traffic signal control at an isolated intersection [1]. OSFI replaces the compositional rule of inference with an optimization problem that maximizes the similarity of truth tables, so the inferred green extension stays close to the ideal behaviour defined by the rule base. Embedded in a three-module controller (NPS, a two-stage GPE with 16 + 13 rules, and SPM), it adapts green time and phase order to real-time queues and arrival rates, giving a flexible cycle length.

According to the paper, OSFI reduces average delay against Mamdani and Sugeno in every phase of both moderate and heavy traffic, while keeping the interpretable, rule-based and low-compute character of fuzzy logic. The improvements are incremental and come with lower exit counts in most phases, and the evaluation covers only one isolated intersection in simulation. The Python prototype demonstrates the workflow and confirms the large benefit of adaptive fuzzy control over fixed-time control, but, as an unoptimized approximation, it does not reproduce OSFI's advantage. Overall, OSFI is a promising first step toward more adaptive yet explainable signal control in support of SDG 11.

7.2 Limitations

- **Single intersection:** no coordination across interconnected intersections, spillback or network effects.
- **Simulation only:** no field validation across intersection geometries, dedicated turn lanes or adaptive coordination.
- **Fixed membership functions:** may not adapt to sudden changes such as accidents or bad weather.
- **Sensor dependence:** requires accurate real-time queue and arrival data; faulty sensors lead to poor timings.
- **Computational cost:** similarity optimization of truth tables can be heavy for complex intersections.
- **Vehicles only:** pedestrians, cyclists, transit priority and left-turn-on-red are not modelled.
- **Prototype:** an approximation with one run per setting and slightly different arrival streams per controller.

7.3 Future Scope

- **Network-wide control:** extend OSFI to multi-intersection corridors with multi-objective models (total travel time, corridor delay, queue balance), possibly through multi-agent or decentralized frameworks.
- **Self-tuning:** adaptive membership functions and rules that respond to sudden changes without manual recalibration.
- **Computational scaling:** faster similarity-optimization algorithms for larger networks.
- **Field pilots:** small-scale pilots and simulation-to-reality transfer before city-wide rollout.
- **Broader scenarios:** multimodal traffic, rural intersections, extreme demand and peak-hour fluctuations.
- **Alternative controllers:** comparison with type-2 fuzzy and neuro-fuzzy systems.
- **Prototype improvements:** implement a true similarity-maximization step, use the paper's rules and membership functions, share one arrival stream across controllers, and repeat each run with 10–30 seeds, reporting mean and standard deviation.

8. Appendix I: Full Python Implementation

The following self-contained script, traffic_simulation.py, implements the four controllers, runs the moderate and heavy traffic scenarios, and produces the prototype results reported in Chapter 6 (results.csv and the graphs). It requires Python 3 with NumPy and Matplotlib (pip install numpy matplotlib) and is run with: python traffic_simulation.py

```
"""
OSFI-inspired traffic signal simulation prototype
-------------------------------------------------
Run:
    python traffic_simulation.py

This educational prototype compares:
    1. Fixed-time
    2. Mamdani-style fuzzy controller
    3. Sugeno-style fuzzy controller
    4. OSFI-inspired similarity-tuned fuzzy controller

Important:
This is NOT an exact reproduction of the cited paper. The presentation does not
provide all membership-function parameters, rules, or the original truth table.
The OSFI-inspired controller here tunes a small rule table against a transparent
target table. For a research-grade reproduction, replace these definitions with
the paper's exact specifications.
"""

from pathlib import Path
import csv
import numpy as np
import matplotlib.pyplot as plt

DIRECTIONS = ["North", "East", "South", "West"]
N = len(DIRECTIONS)

# Simulation settings (seconds)
SIM_TIME = 3600
WARMUP = 300
MIN_GREEN = 8
MAX_GREEN = 35
BASE_GREEN = 10
CLEARANCE = 2                 # simplified all-red clearance between phases
VEHICLES_PER_SECOND = 0.90    # discharge capacity while green
SEED = 42

# Arrival rates (vehicles/second) for moderate and heavy demand.
SCENARIOS = {
    "Moderate": np.array([0.10, 0.12, 0.09, 0.11]),
    "Heavy": np.array([0.18, 0.20, 0.16, 0.19]),
}

OUTPUT_DIR = Path("graphs")
OUTPUT_DIR.mkdir(exist_ok=True)


def clamp(value, low, high):
    return max(low, min(high, value))


def memberships(queue_count, arrival_rate):
    """Simple, transparent membership degrees for Low/Medium/High demand."""
    q = float(queue_count)
    a = float(arrival_rate)

    low_q = clamp((18 - q) / 18, 0, 1)
    high_q = clamp((q - 8) / 22, 0, 1)
    med_q = max(0.0, 1.0 - abs(q - 15) / 15)

    low_a = clamp((0.30 - a) / 0.22, 0, 1)
    high_a = clamp((a - 0.25) / 0.25, 0, 1)
    med_a = max(0.0, 1.0 - abs(a - 0.27) / 0.22)

    return {
        "queue_low": low_q, "queue_medium": med_q, "queue_high": high_q,
        "arrival_low": low_a, "arrival_medium": med_a, "arrival_high": high_a,
    }


def fuzzy_rule_outputs(queue_count, arrival_rate):
    """
    Return weighted rule strengths and output values.
```

```
    Outputs are extension seconds: zero, short, medium, long.
    """
    m = memberships(queue_count, arrival_rate)
    # Rule strengths: AND is minimum. These rules are illustrative.
    strengths = np.array([
        min(m["queue_low"], m["arrival_low"]),
        min(m["queue_low"], m["arrival_medium"]),
        min(m["queue_medium"], m["arrival_low"]),
        min(m["queue_medium"], m["arrival_medium"]),
        min(m["queue_high"], max(m["arrival_medium"], m["arrival_high"])),
        min(m["queue_medium"], m["arrival_high"]),
    ], dtype=float)
    outputs = np.array([0.0, 3.0, 3.0, 7.0, 15.0, 11.0])
    return strengths, outputs


def mamdani_extension(queue_count, arrival_rate):
    """Mamdani-style centroid approximation using sampled output membership shapes."""
    strengths, centers = fuzzy_rule_outputs(queue_count, arrival_rate)
    x = np.linspace(0, 18, 181)
    aggregated = np.zeros_like(x)
    # Triangular output membership functions, clipped by each rule strength.
    widths = [3.0, 4.0, 4.0, 5.0, 5.0, 5.0]
    for strength, center, width in zip(strengths, centers, widths):
        if strength <= 0:
            continue
        triangle = np.maximum(0.0, 1.0 - np.abs(x - center) / width)
        aggregated = np.maximum(aggregated, np.minimum(strength, triangle))
    if aggregated.sum() == 0:
        return 0.0
    return float(np.sum(x * aggregated) / np.sum(aggregated))


def sugeno_extension(queue_count, arrival_rate):
    """Sugeno-style weighted average with crisp rule consequents."""
    strengths, outputs = fuzzy_rule_outputs(queue_count, arrival_rate)
    total = strengths.sum()
    if total <= 1e-9:
        return 0.0
    return float(np.dot(strengths, outputs) / total)


def target_extension(queue_count, arrival_rate):
    """
    Transparent target table used to demonstrate similarity-based tuning.
    It is a heuristic target, NOT the paper's unavailable original truth table.
    """
    demand_score = 0.75 * queue_count + 24.0 * arrival_rate
    if demand_score < 6:
        return 0.0
    if demand_score < 14:
        return 4.0
    if demand_score < 24:
        return 9.0
    return 15.0


def build_similarity_tuned_table():
    """
    Build a compact lookup table. Each cell uses the target output for a
    representative queue/arrival pair. This demonstrates the workflow of
    truth-table construction and rule-output tuning.
    """
    queue_bins = [0, 5, 12, 22, 35, 50]
    arrival_bins = [0.10, 0.20, 0.30, 0.40, 0.50]
    table = {}
    for qi, q in enumerate(queue_bins):
        for ai, a in enumerate(arrival_bins):
            table[(qi, ai)] = target_extension(q, a)
    return queue_bins, arrival_bins, table


OSFI_Q_BINS, OSFI_A_BINS, OSFI_TABLE = build_similarity_tuned_table()


def osfi_extension(queue_count, arrival_rate):
    """OSFI-inspired inference by nearest truth-table cell."""
    qi = int(np.argmin(np.abs(np.asarray(OSFI_Q_BINS) - queue_count)))
    ai = int(np.argmin(np.abs(np.asarray(OSFI_A_BINS) - arrival_rate)))
    return float(OSFI_TABLE[(qi, ai)])


def choose_phase(queues, current_phase, controller):
    """Adaptive controllers choose the longest queue; fixed-time cycles."""
    if controller == "Fixed-time":
        return (current_phase + 1) % N
    # Tie-breaking is deterministic, so results are reproducible.
    return int(np.argmax([len(q) for q in queues]))
```

```
def green_duration(controller, phase, queues, arrival_rates):
    q_len = len(queues[phase])
    rate = arrival_rates[phase]
    if controller == "Fixed-time":
        return 15
    if controller == "Mamdani":
        ext = mamdani_extension(q_len, rate)
    elif controller == "Sugeno":
        ext = sugeno_extension(q_len, rate)
    elif controller == "OSFI-inspired":
        ext = osfi_extension(q_len, rate)
    else:
        raise ValueError(f"Unknown controller: {controller}")
    return int(round(clamp(BASE_GREEN + ext, MIN_GREEN, MAX_GREEN)))


def run_simulation(scenario_name, arrival_rates, controller, seed):
    rng = np.random.default_rng(seed)
    queues = [[] for _ in range(N)]  # each item is vehicle arrival time
    phase = N - 1
    remaining_green = 0
    clearance_left = 0

    waiting_times = []
    queue_sum = np.zeros(N, dtype=float)
    queue_samples = 0
    exits = np.zeros(N, dtype=int)
    total_arrivals = np.zeros(N, dtype=int)
    timeline = []
    queue_history = []

    for t in range(SIM_TIME):
        # Poisson arrivals, independently generated for each approach.
        for i, rate in enumerate(arrival_rates):
            count = int(rng.poisson(rate))
            total_arrivals[i] += count
            for _ in range(count):
                queues[i].append(t)

        # Start a new phase when current green and clearance are finished.
        if remaining_green <= 0 and clearance_left <= 0:
            phase = choose_phase(queues, phase, controller)
            remaining_green = green_duration(controller, phase, queues, arrival_rates)

        # Vehicles leave only from the active green approach.
        if clearance_left <= 0 and remaining_green > 0:
            # Stochastic discharge with a maximum average service capacity.
            capacity = int(rng.poisson(VEHICLES_PER_SECOND))
            for _ in range(min(capacity, len(queues[phase]))):
                arrival_time = queues[phase].pop(0)
                if t >= WARMUP:
                    waiting_times.append(max(0, t - arrival_time))
                if t >= WARMUP:
                    exits[phase] += 1
            remaining_green -= 1

            if remaining_green <= 0:
                clearance_left = CLEARANCE
        else:
            clearance_left -= 1

        if t >= WARMUP:
            queue_sum += np.array([len(q) for q in queues], dtype=float)
            queue_samples += 1
            timeline.append(t)
            queue_history.append([len(q) for q in queues])

    completed = int(exits.sum())
    mean_delay = float(np.mean(waiting_times)) if waiting_times else 0.0
    mean_queue = float(queue_sum.sum() / max(queue_samples * N, 1))
    unfinished = sum(len(q) for q in queues)

    return {
        "scenario": scenario_name,
        "controller": controller,
        "average_delay_s": mean_delay,
        "throughput_vehicles": completed,
        "average_queue_vehicles_per_approach": mean_queue,
        "arrivals": int(total_arrivals.sum()),
        "unfinished_vehicles": int(unfinished),
        "exits_by_direction": exits.tolist(),
        "timeline": np.asarray(timeline),
        "queue_history": np.asarray(queue_history),
    }


def save_results(results):
    csv_path = Path("results.csv")
    fields = [
        "scenario", "controller", "average_delay_s",
```

```
        "throughput_vehicles", "average_queue_vehicles_per_approach",
        "arrivals", "unfinished_vehicles",
    ]
    with csv_path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        writer.writeheader()
        for result in results:
            writer.writerow({key: result[key] for key in fields})
    print(f"\nSaved summary to: {csv_path.resolve()}")


def create_graphs(results):
    controllers = ["Fixed-time", "Mamdani", "Sugeno", "OSFI-inspired"]
    scenarios = list(SCENARIOS.keys())

    for metric, ylabel, filename in [
        ("average_delay_s", "Average waiting delay (seconds)", "average_delay.png"),
        ("throughput_vehicles", "Vehicles discharged after warm-up", "throughput.png"),
        ("average_queue_vehicles_per_approach", "Average queue per approach (vehicles)",
"queue_length.png"),
    ]:
        fig, ax = plt.subplots(figsize=(9, 5))
        x = np.arange(len(controllers))
        width = 0.36
        for j, scenario in enumerate(scenarios):
            values = [
                next(r[metric] for r in results
                     if r["scenario"] == scenario and r["controller"] == c)
                for c in controllers
            ]
            ax.bar(x + (j - 0.5) * width, values, width, label=scenario)
        ax.set_xticks(x)
        ax.set_xticklabels(controllers, rotation=12)
        ax.set_ylabel(ylabel)
        ax.set_title(ylabel + " by controller")
        ax.legend()
        ax.grid(axis="y", alpha=0.25)
        fig.tight_layout()
        path = OUTPUT_DIR / filename
        fig.savefig(path, dpi=160)
        plt.close(fig)
        print(f"Saved graph: {path.resolve()}")

    # Example queue time series for OSFI-inspired controller under heavy traffic.
    example = next(r for r in results
                   if r["scenario"] == "Heavy" and r["controller"] == "OSFI-inspired")
    fig, ax = plt.subplots(figsize=(10, 5))
    for i, direction in enumerate(DIRECTIONS):
        ax.plot(example["timeline"], example["queue_history"][:, i], label=direction)
    ax.set_xlabel("Simulation time (seconds)")
    ax.set_ylabel("Queue length (vehicles)")
    ax.set_title("Queue lengths: OSFI-inspired controller, heavy traffic")
    ax.legend()
    ax.grid(alpha=0.25)
    fig.tight_layout()
    path = OUTPUT_DIR / "queue_over_time.png"
    fig.savefig(path, dpi=160)
    plt.close(fig)
    print(f"Saved graph: {path.resolve()}")


def main():
    controllers = ["Fixed-time", "Mamdani", "Sugeno", "OSFI-inspired"]
    all_results = []

    print("OSFI-inspired Four-Way Traffic Signal Simulation")
    print("=" * 58)
    print(f"Simulation duration: {SIM_TIME} s; warm-up excluded from delay/queue metrics: {WARMUP}
s")
    print("This is a prototype; do not present its output as the paper's reported results.\n")

    for scenario_index, (scenario, rates) in enumerate(SCENARIOS.items()):
        print(f"\nScenario: {scenario} | arrival rates (vehicles/s): {rates.tolist()}")
        for controller_index, controller in enumerate(controllers):
            # Same seed per scenario for repeatable, comparable arrival streams.
            result = run_simulation(
                scenario, rates, controller,
                seed=SEED + scenario_index * 100 + controller_index
            )
            all_results.append(result)
            print(
                f"{controller:15s} | delay={result['average_delay_s']:7.2f} s"
                f" | throughput={result['throughput_vehicles']:5d}"
                f" | avg queue={result['average_queue_vehicles_per_approach']:6.2f}"
                f" | unfinished={result['unfinished_vehicles']:5d}"
            )

    save_results(all_results)
    create_graphs(all_results)
    print("\nDone. Open results.csv and the PNG files in the graphs folder.")
```

```
    print("For a stronger experiment, repeat each run with 10-30 seeds and report mean ± standard
deviation.")


if __name__ == "__main__":
    main()
```

9. References

[1] Esmaeili, M., Anjomshoae, A., Shahsavari-Pour, N., Srisurin, P. & Banomyong, R. An optimization similarity fuzzy inference method for traffic signal control at an isolated intersection. *Multimodal Transportation*, 100234 (2025). https://doi.org/10.1016/j.multra.2025.100234

[2] Eom, M. & Kim, B.-I. The traffic signal control problem for intersections: a review. *European Transport Research Review* 12(1), 50 (2020).

[3] Pappis, C. P. & Mamdani, E. H. A fuzzy logic controller for a traffic junction. *IEEE Transactions on Systems, Man, and Cybernetics* 7(10), 707–717 (1977).

[4] Webster, F. V. Traffic signal settings. *Road Research Technical Paper* No. 39, Road Research Laboratory (1958).

[5] Roess, R. P., Prassas, E. S. & McShane, W. R. *Traffic Engineering*. Pearson/Prentice Hall (2004).

[6] Wei, H., Zheng, G., Gayah, V. & Li, Z. A survey on traffic signal control methods. arXiv:1904.08117 (2019).

[7] Cakici, Z. & Murat, Y. S. A differential evolution algorithm-based traffic control model for signalized intersections. *Advances in Civil Engineering* 2019, 7360939 (2019).

[8] Niittymäki, J. & Pursula, M. Signal control using fuzzy logic. *Fuzzy Sets and Systems* 116(1), 11–22 (2000).

[9] Murat, Y. S. & Gedizlioglu, E. A fuzzy logic multi-phased signal control model for isolated junctions. *Transportation Research Part C* 13(1), 19–36 (2005).

[10] Collotta, M., Lo Bello, L. & Pau, G. A novel approach for dynamic traffic lights management based on wireless sensor networks and multiple fuzzy logic controllers. *Expert Systems with Applications* 42(13), 5403–5415 (2015).

[11] Bi, Y., Srinivasan, D., Lu, X., Sun, Z. & Zeng, W. Type-2 fuzzy multi-intersection traffic signal control with differential evolution optimization. *Expert Systems with Applications* 41 (2014).

[12] Jovanović, A., Kukić, K., Stevanović, A. & Teodorović, D. Restricted crossing U-turn traffic control by interval Type-2 fuzzy logic. *Expert Systems with Applications* 211, 118613 (2023).

[13] Ali, M. E. M., Durdu, A., Celtek, S. A. & Yilmaz, A. An adaptive method for traffic signal control based on fuzzy logic with Webster and modified Webster formula using SUMO traffic simulator. *IEEE Access* 9, 102985–102997 (2021).

[14] Lin, H., Han, Y., Cai, W. & Jin, B. Traffic signal optimization based on fuzzy control and differential evolution algorithm. *IEEE Transactions on Intelligent Transportation Systems* 24(8), 8555–8566 (2023).

[15] Chiou, Y.-C. & Huang, Y.-F. Stepwise genetic fuzzy logic signal control under mixed traffic conditions. *Journal of Advanced Transportation* 47(1), 43–60 (2013).

[16] Kumar, N., Rahman, S. S. & Dhakad, N. Fuzzy inference enabled deep reinforcement learning-based traffic light control for intelligent transportation system. *IEEE Transactions on Intelligent Transportation Systems* 22(8), 4919–4928 (2021).

[17] Bellman, R. E. & Zadeh, L. A. Decision-making in a fuzzy environment. *Management Science* 17(4), B141–B164 (1970).

[18] Deng, G. & Jiang, Y. Fuzzy reasoning method by optimizing the similarity of truth-tables. *Information Sciences* 288, 290–313 (2014).

[19] Esmaeili, M. & Eslami, E. Intuitionistic fuzzy reasoning using the method of optimizing the similarity of truth tables. *Soft Computing* 23(18), 8765–8774 (2019).

[20] Soh, A. C., Marhaban, M. H., Khalid, M. & Yusof, R. Modelling and optimisation of a traffic intersection based on queue theory and Markov decision control methods. In *First Asia International Conference on Modelling & Simulation (AMS'07)*, 478–483 (2007).

[21] Zadeh, L. A. Fuzzy sets. *Information and Control* 8(3), 338–353 (1965).

[22] Nguyen, H. T., Walker, C. & Walker, E. A. *A First Course in Fuzzy Logic*. Chapman and Hall/CRC (2018).

[23] Zadeh, L. A. Similarity relations and fuzzy orderings. *Information Sciences* 3(2), 177–200 (1971).

[24] Zadeh, L. A. Outline of a new approach to the analysis of complex systems and decision processes. *IEEE Transactions on Systems, Man, and Cybernetics* 3(1), 28–44 (1973).